

PersistHotStuff: Production-Oriented HotStuff BFT Consensus

Saanvi Shet - 612203164

Tejas Kolhe - 612203097

Dr.Vahida Attar

March 2026



05-03-2026

Department of Computer Science & Engineering and Information Technology
School of Computational Sciences
COEP Technological University

Motivation: Why BFT for critical replicated services?

- **Byzantine faults:** A replica performing arbitrary/malicious behavior: lie, send conflicting messages to different peers (equivocation), omit messages, replay old messages, or follow a different/buggy protocol version.
- **Safety required under adversarial conditions:** Safety in distributed systems ensures that "bad things" never happen, maintaining system correctness, data integrity, and consistency even during failures
- **Consensus** is needed for replicated state machines: process where multiple independent nodes (computers) agree on a single data value or state even when some nodes fail or network issues occur
- **Deployments face crashes and restarts too.** It risks forgetting critical protocol state, which can break safety after restart if the node behaves like it's "fresh."



Problem Statement

- **HotStuff** is a leader-based Byzantine Fault Tolerant (BFT) consensus protocol known for its linear view changes, high responsiveness, and safety in partially synchronous networks.
- However there exist four practical limitations

Limitation	Impact
L1 Persistence + crash recovery	Safety breaks after restart
L2 Dynamic membership	Can't add/remove replicas safely
L3 Pluggable state machine	Hard to integrate real applications
L4 Enhanced pacemaker	Liveness fragile under delays/failures



Objectives

- O1: Write Ahead Log (WAL) + snapshots + deterministic recovery
- O2: Safe membership transitions
- O3: Clean, pluggable state-machine API
- O4: Robust pacemaker (timeouts, leader rotation)

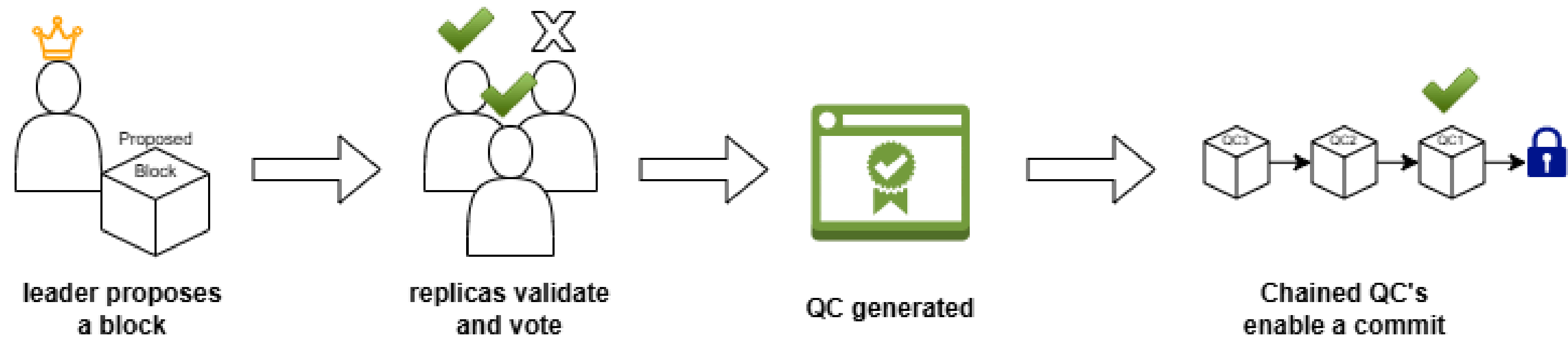
Lx	Ox	Success criteria
L1	O1	Restart yields same committed chain/state
L2	O2	Reconfig does not violate quorum safety
L3	O3	App logic swappable without consensus rewrite
L4	O4	Progress continues despite of leader/network stalls



Literature Review

Protocol	Paper	Findings
Hotstuff	<u>"HotStuff: BFT Consensus in the Lens of Blockchain"</u>	• Reduction of communication complexity to $O(n)$ per decision (vs. $O(n^2)$ in PBFT).' • Proof that a 3-chain commit rule is sufficient and necessary for safety.
DynaNet	<u>"DynaNet: A Dynamic BFT Consensus Framework"</u>	• Explores reconfiguration in BFT systems and proposes flexible membership management under Byzantine faults.
RAFT	<u>"The Raft Consensus Algorithm"</u>	• Raft uses Write-Ahead Logging (WAL) and periodic snapshots to ensure that nodes can reboot without losing committed entries
Tendermint	<u>"Tendermint ABCI (Application Blockchain Interface)"</u>	• Allows arbitrary applications to connect over a message-based interface

Broad Design - Part 1 (Process Flow)



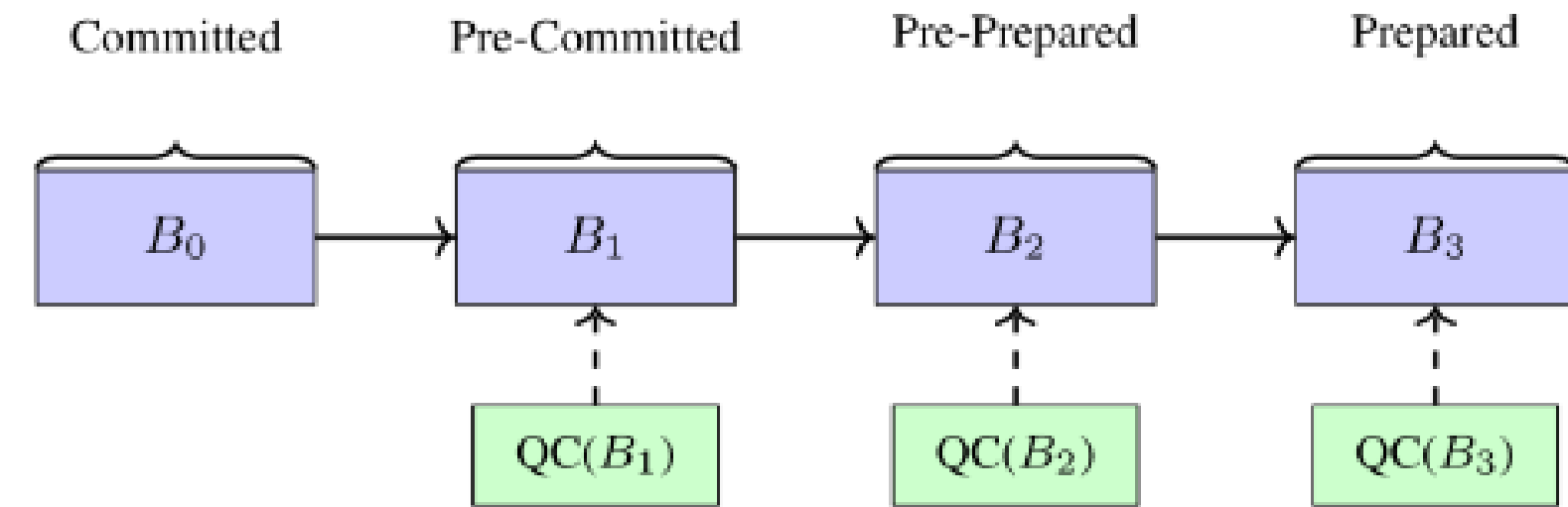
View	Identifies a leader's term
Leader	Proposes blocks for the view
Replica	Votes on a proposed block
QC	Collection of signed votes from replicas



Part 2 - (Consensus Logic)

Constraints -

- f - Number of byzantine replicas
- n - Total number of replicas
- Quorum size - $2f + 1$
- Total number of replicas $\geq 3f + 1$



3-Chain Commit: B_0 committed when $B_0 \leftarrow B_1 \leftarrow B_2$ justified by QCs

Fig. 1: HotStuff 3-Chain Commit Rule Visualization

QC1 $\rightarrow 2*f + 1$ (quorum size)

QC2 $\rightarrow 2*f + 1$ (quorum size)

QC3 $\rightarrow 2*f + 1$ (quorum size)

Quorum intersection (QC1 $\cap$ QC2) $\rightarrow f + 1$ (assuming $\geq 3*f+1$ number of replicas)

This guarantees at least 1 honest node once a chain of 3 QC's is formed

Part 2 - (System Architecture)

- **Per replica core logic:** HotStuff + QC handling
- **Networking transport:** Pluggable abstraction
- **Persistence:** WAL + snapshots; Recovery module
- **Membership manager:** configuration changes
- **Pacemaker:** Liveness
- **Crypto:** signatures and QC verification

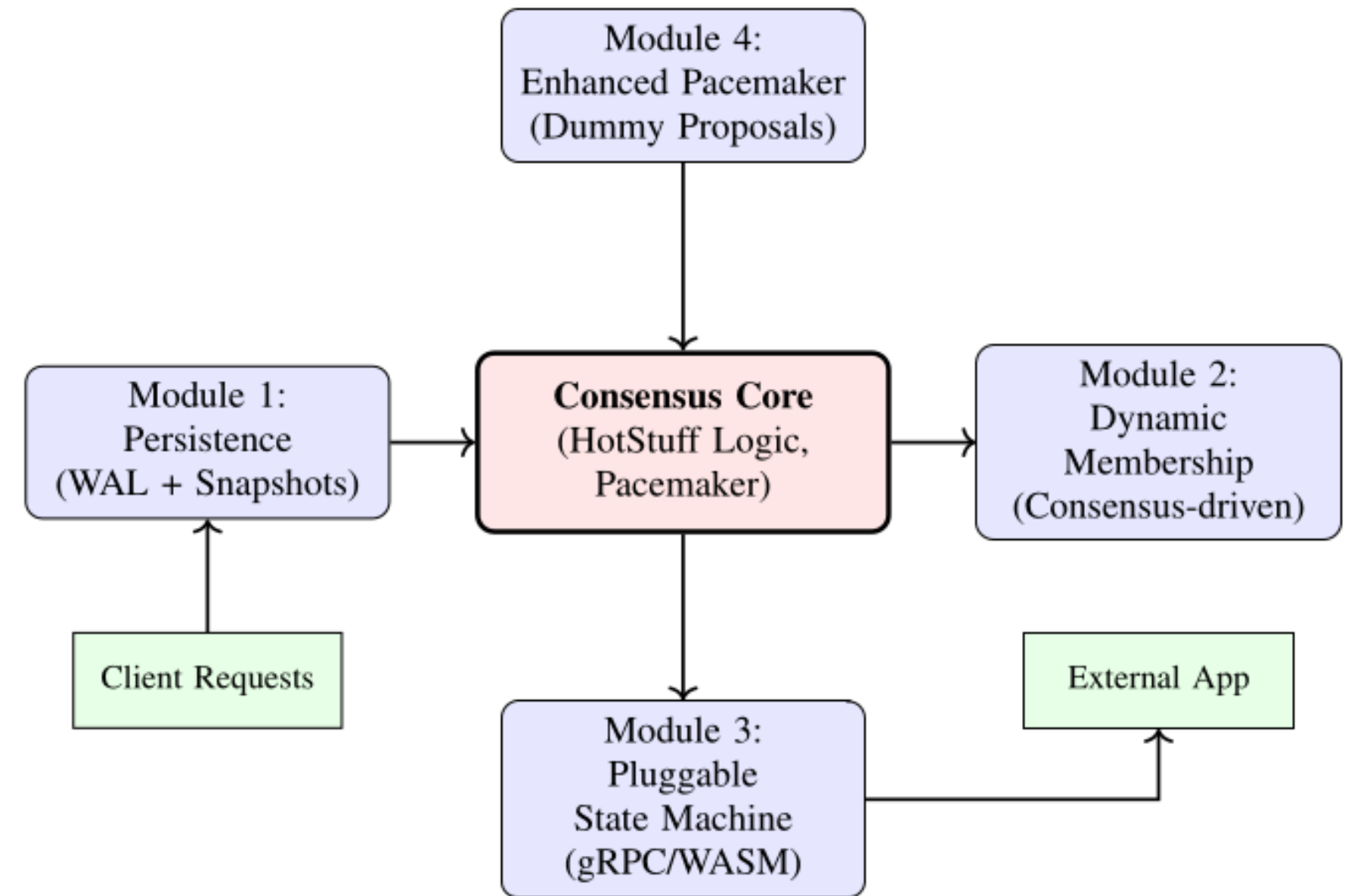


Fig. 2: PersistHotStuff Architecture Diagram

M1 - Persistence/Recovery

- WAL records **proposals, QCs, commits** deterministically
- **Snapshot** compacts state; speeds recovery
- **Recovery replays WAL**; restores safe committed state

```
Crash
|
Load snapshot
|
Replay WAL sequentially
|
Restore chain & state (same commits)
```

Each WAL entry includes a file header and the entry frame.

Offset	Size	Field	Description
0	4	Magic Number	0x48534C57 (“HSLW” = HotStuff WAL)
4	2	Version	Format version (e.g., 0x0001)
6	1	Replica ID	Replica that owns this log
14	8	Created Time	Unix timestamp (ms)
22	32	Header Checksum	SHA256 of header

TABLE I: File header layout

Entry Frame		
Length (4 B)	Checksum (4 B)	Payload

TABLE II: Log entry frame format

M2 - Dynamic Membership

- Config changes via explicit “**reconfig tx**”
- **Safety**: quorum intersections across configs

View 5: Leader proposes "AddValidator(R4)" ← just a command in a block

View 6: Block gets QC

View 7: Block gets QC

View 8: 3-chain satisfied → command committed

↓

NOW switch to new config (n=5, f=1)

M3 - Pluggable State Machine

A clean API boundary for enhanced flexibility

- Separates **consensus engine** from **application logic**
- **Enables multiple applications** without rewriting protocol
- Supports snapshot/export for recovery

[Clients] → [State Machine API]



M4 - Enhanced Pacemaker

Addressing liveness in dynamic network conditions

- **Leader failure stalls progress** without view changes
- Variable **network delay** causes premature timeouts
- Need **adaptive timeouts + leader rotation rules**

View	Timeout	Trigger	Action
v	t0	leader stalls	enter new view
v+1	t1>t0	network delay	rotate leader



Implementation Methodology - Development Strategy

We will be using an **incremental model** to add the features in phases

Key Aspects of the Methodology:

- **Rust Modules:** Each module is aligned with specific protocol components for modularity and maintainability.
Enables independent development and testing of features.
- **Phase Milestones/Modules :**
 - Core: Basic HotStuff implementation.
 - Persistence: Write-Ahead Logging (WAL) and snapshots for crash recovery.
 - Membership: Dynamic join/leave operations using quorum certificates.
 - Pluggable State Machine: gRPC-based interface for extensible state machines.
 - Pacemaker: Dummy proposals for improved liveness under low client load/leader stalls.



Implementation Methodology - Testing Strategy

Comprehensive verification for reliability and correctness

- Unit tests: crypto, data types, WAL logic
- Protocol tests: QC formation, commit rule, safety invariants
- Network simulation: multi-replica behavior under delays
- Crash recovery: restart correctness across cycles
- Docker-based multi-node system tests

Test type	What it proves
Unit tests	Correctness of individual components
Protocol tests	Safety (no conflicting commits)
Network simulation	Liveness under network conditions
Crash recovery	Durability and consistency



Results: Test & Simulation Summary

- | | |
|---|---|
| <ul style="list-style-type: none">• Persistence (18) —<ul style="list-style-type: none">◦ WAL create/append/read/corrupt/truncate◦ Snapshot save/load/checksum/cleanup◦ Crash recovery from fresh start, WAL-only, snapshot+WAL, and out-of-order replays• QC Formation (3) —<ul style="list-style-type: none">◦ $\geq 2f+1$ signatures enforced◦ Ed25519 votes aggregate into valid QC◦ high_qc updates correctly• 3-Chain Commit (4) —<ul style="list-style-type: none">◦ Full chain commits◦ Partial chains blocked◦ Multi-block chains commit in order◦ No double-commits | <ul style="list-style-type: none">• Proposal Phase (4) —<ul style="list-style-type: none">◦ Only round-robin leader can propose◦ Valid proposals accepted◦ Under-quorum QCs rejected• Pacemaker (7) —<ul style="list-style-type: none">◦ Timeout increments view◦ Clears vote pool◦ Leader rotation verified across 8 views◦ Timer resets on proposal and commit• Network (3) —<ul style="list-style-type: none">◦ Send/receive round-trip◦ Broadcast reaches exactly $n-1$ replicas |
|---|---|



Roadmap & Success Metrics

Charting the path forward and key success criteria

Remaining Work + 10-Week Timeline

Week 1-3: finalize Dynamic Membership integration
Week 4-5: integrating State Machines
Week 6-7: pacemaker tuning + liveness tests
Week 7-8: edge cases + reconfig tests
Week 9-10: final demo + report polishing

Success Metrics Checklist

Metric	Pass Criteria
Safety	No conflicting commits
Recovery	Restart preserves state
Liveness	Commits continue
Reconfig	Join/leave quorum safety



05-03-2026

Department of Computer Science & Engineering and Information Technology
School of Computational Sciences
COEP Technological University

References

- [1] M. Yin, D. Malkhi, M. K. Reiter, G. G. Gueta, and I. Abraham, "HotStuff: BFT Consensus in the Lens of Blockchain,"
- [2] M. Castro and B. Liskov, "Practical Byzantine Fault Tolerance,"
- [3] D. Ongaro and J. K. Ousterhout, "In Search of an Understandable Consensus Algorithm,"
- [4] D. Kang et al., "HotStuff-1: Linear Consensus with One-Phase Speculation,"
- [5] I. Abraham, D. Malkhi, K. Nayak, L. Ren, and M. Yin, "Sync-HotStuff: Simple and Practical Synchronous State Machine Replication,"
- [6] H. Guo et al., "DynaNet: A Dynamic BFT Consensus Framework"
- [7] StanfordCS244B, "HotStuff Implementation and Advice"
- [8] J. Kwon, "Tendermint: Consensus Without Mining"
- [9] "gRPC: A High Performance, Open Source Universal RPC Framework."
- [10] C. Dwork, N. Lynch, and L. Stockmeyer, "Consensus in the Presence of Partial Synchrony."
- [11] Brendel, Jacqueline and Cremers, Cas and Jackson, Dennis and Zhao, Mang, "The Provable Security of Ed25519: Theory and Practice"
- [12] Chaudhuri Kaustuv, Doligez Damien, Lamport Leslie, Merz Stephan "A TLA+ proof system"



Thank You!

Any Questions?



05-03-2026

Department of Computer Science & Engineering and Information Technology
School of Computational Sciences
COEP Technological University